

Supplementary Requirements Documentation

<https://github.com/DanielJ0131/interactive-house>

Interactive House - Subgroup 2: Units

Revision History.....	1
Supplementary Requirements List.....	2
Supplementary Requirements Descriptions.....	3
1. General.....	3
S1. Usability, Accessibility & Independence.....	3
S2. Reliability - Synchronized State.....	3
S3. Performance - Interaction Latency.....	3
S4. User Interface - Dynamic Scalability.....	4
S5. Hardware Requirements - Multi-modal Input.....	4
S6. Design Language.....	4
S7. Implementation Platform.....	4
S8. Local Persistence – Offline Mode Mock-up.....	4
2. Mobile.....	4
S9. Mobile Performance - Native Thread Responsiveness.....	4
S10. Mobile Reliability - Connectivity Transition (WiFi/Cellular).....	5
S11. Mobile Resource Management - Battery Efficiency.....	5
S12. Mobile Accessibility - Screen Reader & Voiceover.....	5
3. Web.....	5
S13. Web Performance - Interaction Responsiveness.....	5
S14. Web Reliability - Session & Connection Stability.....	5
S15. Web Accessibility - Browser Assistive Support.....	6
S16. Web Responsive Layout - Multi Screen Adaptation.....	6
S17. Web Accessibility – Color Blind Mode – Desirable.....	6
S18. Web Theme Persistence.....	7
Figures.....	7
Figure 1: User perception of performance delays [2].....	7
References.....	7

Revision History

Date	Version	Description	Author
2026-02-04	1.0	Initial revision (Add S1 – S8)	Daniel Jönsson
2026-02-17	1.1	Add Supplementary Requirements for Mobile (S9-S12)	Daniel Jönsson
2026-02-28	1.2	Add Supplementary Requirements for Web (S13-S16)	Mustafa Al-Bayati
2026-03-15	1.3	Update S13 - S14	Mustafa Al-Bayati
2026-04-21	1.4	Add S17 – S18 for Web, Refactoring, Add Table of Contents, Add References to S9 Performance, Add Figure 1	Patrick Nordahl & Daniel Jönsson
2026-05-07	1.5	Add progress to all supplementary requirements	Daniel Jönsson & Patrick Nordahl
2026-05-25	1.6	Add Github Link	Daniel Jönsson

Supplementary Requirements List

Supplementary Requirement Name	Priority	Progress
S1. Usability, Accessibility & Independence	Essential	100%
S2. Reliability - Synchronized State	Essential	100%
S3. Performance - Interaction Latency	Essential	100%
S4. User Interface - Dynamic Scalability	Essential	100%
S5. Hardware Requirements - Multi-modal Input	Essential	100%
S6. Design language	Essential	100%
S7. Implementation Platform	Desirable	100%
S8. Local Persistence – Offline Mode Mock-up	Optional	0%

S9. Mobile Performance - Native Thread Responsiveness	Essential	100%
S10. Mobile Reliability - Connectivity Transition (WiFi/Cellular)	Essential	100%
S11. Mobile Resource Management - Battery Efficiency	Optional	0%
S12. Mobile Accessibility - Screen Reader & Voiceover	Essential	50%
S13. Web Performance - Interaction Responsiveness	Essential	100%
S14. Web Reliability - Session & Connection Stability	Essential	100%
S15. Web Accessibility - Browser Assistive Support	Essential	100%
S16. Web Responsive Layout - Multi Screen Adaptation	Desirable	100%
S17. Web Accessibility – Color Blind Mode	Essential	100%
S18. Web Theme Persistence	Essential	100%

Supplementary Requirements Descriptions

1. General

S1. Usability, Accessibility & Independence

The interface must prioritize self-control and accessibility for users with high grades of disabilities. Controls must be learnable within a minute, enabling patients to access home functionalities like turning on lights, opening windows, doors, or a coffee machine independently.

S2. Reliability - Synchronized State

The unit must maintain a stable connection with the House Server. It must reflect the present devices for observation without locking the program, ensuring the user always sees an accurate list of controllable devices provided by the server.

S3. Performance - Interaction Latency

The interface must be highly responsive to provide a ubiquitous feel. Commands sent from the unit like toggling a light, and the rendering of new UI components uploaded from the server should occur within 500 milliseconds to ensure real-time control.

S4. User Interface - Dynamic Scalability

The Unit must be capable of rendering unique UI software components sent from the server for any plugged-in device. To maintain simplicity for disabled users, the maximum menu depth for any device control should not exceed 3 levels.

S5. Hardware Requirements - Multi-modal Input

Mobile Units: Must support touchscreens like iPhone and Android, speech recognition, and gestures (at screen or in the air).

Web Units: Must be compatible with standard laptop displays and mouse utilities.

Other: Support for components detecting brain activities for severely disabled users.

S6. Design Language

The unit architecture and interaction logic must be described through specific UML diagrams, including Use Case, Class, Interaction Sequence, and State diagrams.

S7. Implementation Platform

We had an alternative, like using Android Studio + Jetpack Compose specifically for mobile devices, but this only works on Android. The Units will therefore be implemented using React Native and Expo. This choice is a non-functional constraint to ensure a uniform model of communication. By using a JavaScript/TypeScript-based stack, the subgroup can fulfil the requirement to dynamically upload, and render UI components received from the server, while maintaining cross-platform support for both mobile touchscreens and web-based mouse interfaces. It may also have support for iOS in the future.

S8. Local Persistence – Offline Mode Mock-up

The Unit may implement a demo mode or cached state. If the House Server is temporarily unreachable, the Unit should show the last known state of the house rather than a blank screen.

2. Mobile

S9. Mobile Performance - Native Thread Responsiveness

The mobile unit must maintain a consistent frame rate (target 60 FPS) to ensure a fluid experience for users with motor impairments. The goal of 60 frames per second (FPS) is the global standard for native-like smoothness. "A compelling reason to use React Native... is to achieve at least 60 frames per second and provide a native look and feel to your apps." [1] This equates to approximately 16.6ms per frame. "iOS and Android devices display at least 60 frames per second, which gives you and the UI system at most 16.67ms to do all of the work needed... If you are unable to do the work... you will 'drop a frame' and the UI will appear unresponsive." [1] Because React Native operates with separate threads for JavaScript logic and Native UI

rendering, long-running tasks (such as processing complex device lists from the server) must not block the main UI thread. Any interaction, such as a button press or a navigation transition, must initiate a visual response within 500ms to provide immediate responsiveness to the user. By setting a 500ms limit, we are effectively requiring that the app completes its "visual response" (like a navigation transition or a loading spinner) at the midpoint of that window. This ensures the user never reaches the 1000ms (1 second) mark, where they statistically begin to lose focus. [2]

S10. Mobile Reliability - Connectivity Transition (WiFi/Cellular)

As mobile units are often in motion, the application must gracefully handle the handover between local WiFi and cellular data without session termination.

Utilizing expo-network, the Unit must detect connection changes. If the House Server becomes unreachable during a transition, the UI must display a "Reconnecting" status overlay while retaining the last known state (as per S8) to prevent user confusion or accidental double-triggering of commands.

S11. Mobile Resource Management - Battery Efficiency

Since the mobile unit is a life assistance tool, it must minimize power consumption to ensure all day availability. The app should utilize expo-battery to monitor energy levels. When the device's battery falls below 20%, the Unit should automatically reduce the frequency of background state polling from the House Server and disable non-essential UI animations to preserve remaining power for critical home controls.

S12. Mobile Accessibility - Screen Reader & Voiceover

The mobile interface must be fully navigable via system level assistive technologies (iOS VoiceOver and Android TalkBack) to support users with visual or severe motor impairments. All dynamic UI components rendered from the server must be injected with appropriate `accessible={true}`, `accessibilityLabel`, and `accessibilityRole` props. For example, a "Light Switch" component must be announced by the device as a toggleable element with its current state ("On/Off"), rather than just a generic image or button.

3. Web

S13. Web Performance - Interaction Responsiveness

When a user clicks a button, toggles a device, or moves a slider, the interface must immediately show a visual change (for example, button highlight, loading indicator, or updated state).

Device updates received from the House Server must appear automatically without requiring the user to refresh the page. This is implemented using Firebase Firestore real-time listeners, ensuring that device states and sensor values are synchronized with the interface instantly.

S14. Web Reliability - Session & Connection Stability

The Web Unit must keep users logged in securely while they are using the system using Firebase Authentication. Protected pages such as the Hub must not be accessible without authentication.

If the connection to the cloud database is lost, the Web Unit must:

- Show a clear connection status message (for example: Disconnected or Reconnecting)
- Prevent sending duplicate commands
- Attempt automatic reconnection

During reconnection, the system should continue displaying the last known device state to avoid confusing the user.

S15. Web Accessibility - Browser Assistive Support

The Web interface must be usable by people with disabilities.

All buttons, switches, sliders, and other controls must:

- Be usable with a keyboard (not only a mouse)
- Use proper HTML structure
- Include accessibility labels when needed
- Have good color contrast for readability

Dynamic components received from the server must also follow these accessibility rules. For example, a light switch must be announced as a switch with its current state "On" or "Off" not just as a simple button.

S16. Web Responsive Layout - Multi Screen Adaptation

The Web Unit must work properly on different screen sizes, including:

- Desktop computers
- Laptops
- Tablets
- Mobile browsers

The layout must adjust automatically so that buttons and controls remain clear and easy to use. No important functionality should be hidden or broken on smaller screens.

The design should remain consistent with the Mobile Unit while optimizing mouse and keyboard interaction.

S17. Web Accessibility – Color Blind Mode – Desirable

The Web Unit should provide an optional color-blind mode to improve accessibility for users with visual impairments.

This functionality is already supported in the Mobile Unit and should be extended to the Web interface to ensure consistency across platforms.

The color-blind mode should adjust the interface color palette to improve contrast and distinguishability between UI elements and should not rely solely on color to convey important information.

S18. Web Theme Persistence

The Web Unit should remember the user's selected theme (e.g., light mode, dark mode, or accessibility modes) during the session or across visits using local storage or user preferences.

This improves usability and ensures a consistent experience without requiring the user to reselect their preferred settings.

Figures

Focus on the user

Make users the focal point of your performance effort. The table below describes key metrics of how users perceive performance delays:

User perception of performance delays

0 to 16 ms	Users are exceptionally good at tracking motion, and they dislike it when animations aren't smooth. They perceive animations as smooth so long as 60 new frames are rendered every second. That's 16 ms per frame, including the time it takes for the browser to paint the new frame to the screen, leaving an app about 10 ms to produce a frame.
0 to 100 ms	Respond to user actions within this time window and users feel like the result is immediate. Any longer, and the connection between action and reaction is broken.
100 to 1000 ms	Within this window, things feel part of a natural and continuous progression of tasks. For most users on the web, loading pages or changing views represents a task.
1000 ms or more	Beyond 1000 milliseconds (1 second), users lose focus on the task they are performing.
10000 ms or more	Beyond 10000 milliseconds (10 seconds), users are frustrated and are likely to abandon tasks. They may or may not come back later.

Figure 1: User perception of performance delays [2]

References

[1] "Performance Overview · React Native." Accessed: Apr. 21, 2026. [Online]. Available: <https://reactnative.dev/docs/performance>

[2] "Measure performance with the RAIL model | Articles," web.dev. Accessed: Apr. 21, 2026. [Online]. Available: <https://web.dev/articles/rail>